

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part 1 — Build Your Own Practice Schema

I have created a database schema (Food Delivery) and then created 4 tables which are customer, restaurants, orders, accounts.

Then I have inserted the records in the all table according to the task condition.



Schema SQL

```
1 CREATE DATABASE FoodDelivery;
2 USE FoodDelivery;
3
4 CREATE TABLE customers (
5   id INT PRIMARY KEY AUTO_INCREMENT,
6   name VARCHAR(100) NOT NULL,
7   email VARCHAR(100) UNIQUE,
8   signup_date DATE
9 );
10
11 CREATE TABLE restaurants (
12   id INT PRIMARY KEY AUTO_INCREMENT,
13   name VARCHAR(100),
14   cuisine VARCHAR(50)
15 );
16
17 CREATE TABLE orders (
18   id INT PRIMARY KEY AUTO_INCREMENT,
19   customer_id INT,
20   restaurant_id INT,
21   total DECIMAL(10,2),
22   status VARCHAR(20),
23   created_at DATETIME,
24
25   FOREIGN KEY (customer_id) REFERENCES customers(id),
26   FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)
27 );
28
29 CREATE TABLE accounts (
30   id INT PRIMARY KEY AUTO_INCREMENT,
31   name VARCHAR(50),
32   balance DECIMAL(10,2)
33 );
```

```
INSERT INTO customers(name,email,signup_date) VALUES
('Rahul','rahul@gmail.com','2024-01-10'),
('Amit','amit@gmail.com','2024-01-12'),
('Neha','neha@gmail.com','2024-01-18'),
('Priya','priya@gmail.com','2024-01-22'),
('Karan','karan@gmail.com','2024-01-25'),
('Riya','riya@gmail.com','2024-02-01'),
('Sneha','sneha@gmail.com','2024-02-05'),
('Vivek','vivek@gmail.com','2024-02-10'),
('Ankit','ankit@gmail.com','2024-02-15'),
('Pooja','pooja@gmail.com','2024-02-18'),
('Rohan','rohan@gmail.com','2024-02-20'),
('Deep','deep@gmail.com','2024-02-22'),
('Nisha','nisha@gmail.com','2024-03-01'),
('Mehul','mehul@gmail.com','2024-03-05'),
('Komal','komal@gmail.com','2024-03-10'),
('Arjun','arjun@gmail.com','2024-03-12'),
('Rakesh','rakesh@gmail.com','2024-03-15'),
('Sonal','sonal@gmail.com','2024-03-18'),
('Bhavik','bhavik@gmail.com','2024-03-20'),
('Jay','jay@gmail.com','2024-03-25');
```

```
INSERT INTO restaurants(name,cuisine) VALUES
('Pizza Hub','Italian'),
('Spice Villa','Indian'),
('Burger Point','Fast Food'),
('Dragon Bowl','Chinese'),
('Taco Land','Mexican'),
('Food Factory','Multi Cuisine'),
('South Treat','South Indian'),
('Royal Biryani','Mughlai'),
('Healthy Bites','Healthy'),
('Cafe Aroma','Cafe');
```

```
INSERT INTO accounts(name,balance)
VALUES
('Alice',10000),
('Bob',7000);
```

```
INSERT INTO orders(customer_id,restaurant_id,total,status,created_at) VALUES
(1,1,450,'completed','2024-04-01 12:00:00'),
(2,3,250,'pending','2024-04-01 12:15:00'),
(3,2,800,'completed','2024-04-01 12:30:00'),
(4,4,650,'cancelled','2024-04-01 13:00:00'),
(5,5,300,'completed','2024-04-01 13:15:00'),
(6,6,900,'completed','2024-04-01 13:30:00'),
(7,7,420,'pending','2024-04-01 14:00:00'),
(8,8,700,'completed','2024-04-01 14:20:00'),
(9,9,380,'completed','2024-04-01 14:40:00'),
(10,10,550,'completed','2024-04-01 15:00:00'),
(11,2,650,'completed','2024-04-02 10:00:00'),
(12,4,450,'completed','2024-04-02 10:30:00'),
(13,5,320,'pending','2024-04-02 11:00:00'),
(14,1,800,'completed','2024-04-02 11:30:00'),
(15,3,220,'completed','2024-04-02 12:00:00'),
(16,7,760,'completed','2024-04-02 12:30:00'),
(17,8,900,'cancelled','2024-04-02 13:00:00'),
```

```
(1,2,510,'completed','2024-04-02 13:20:00'),
(2,6,640,'completed','2024-04-02 13:40:00'),
(3,5,430,'completed','2024-04-02 14:00:00'),
(4,4,550,'pending','2024-04-03 10:00:00'),
(5,3,300,'completed','2024-04-03 10:30:00'),
(6,2,450,'completed','2024-04-03 11:00:00'),
(7,1,800,'completed','2024-04-03 11:30:00'),
(8,10,520,'completed','2024-04-03 12:00:00'),
(9,9,250,'completed','2024-04-03 12:20:00'),
(10,8,680,'pending','2024-04-03 12:40:00'),
(11,7,900,'completed','2024-04-03 13:00:00'),
(12,6,700,'completed','2024-04-03 13:20:00'),
(13,5,420,'completed','2024-04-03 13:40:00'),
(14,4,320,'completed','2024-04-04 10:00:00'),
(15,5,460,'completed','2024-04-04 10:30:00'),
(16,6,610,'pending','2024-04-04 11:00:00'),
(17,7,700,'completed','2024-04-04 11:30:00'),
(1,8,950,'completed','2024-04-04 12:00:00'),
(2,9,330,'completed','2024-04-04 12:20:00'),
(3,10,280,'completed','2024-04-04 12:40:00'),
(4,1,770,'cancelled','2024-04-04 13:00:00'),
(5,2,650,'completed','2024-04-04 13:20:00'),
(6,3,520,'completed','2024-04-04 13:40:00');
```

Output:

Results

There are no results to be displayed.

Then we verify with Select statement to check the records inside the table

Query: **Select * from customers;**

Results

Query #1 Execution time: 0.32ms

id	name	email	signup_date
1	Rahul	rahul@gmail.com	2024-01-10
2	Amit	amit@gmail.com	2024-01-12
3	Neha	neha@gmail.com	2024-01-18
4	Priya	priya@gmail.com	2024-01-22
5	Karan	karan@gmail.com	2024-01-25
6	Riya	riya@gmail.com	2024-02-01
7	Sneha	sneha@gmail.com	2024-02-05
8	Vivek	vivek@gmail.com	2024-02-10
9	Ankit	ankit@gmail.com	2024-02-15
10	Pooja	pooja@gmail.com	2024-02-18

Our Task 1 is completed according to the given conditions and rules.

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```
SELECT * FROM accounts;
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name = 'Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name = 'Bob';

ROLLBACK;
SELECT * FROM accounts;
```

Output:

Results

Query #1 Execution time: 0.19ms

id	name	balance
1	Alice	10000.0
2	Bob	7000.0

Query #2 Execution time: 0.06ms

There are no results to be displayed.

Query #3 Execution time: 0.39ms

There are no results to be displayed.

Query #4 Execution time: 0.21ms

There are no results to be displayed.

Query #5 Execution time: 0.15ms

There are no results to be displayed.

Query #6 Execution time: 0.16ms

id	name	balance
1	Alice	10000.0
2	Bob	7000.0

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name='Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name='Bob';

COMMIT;
SELECT * FROM accounts;
```

Output:

Query #5 **Execution time: 0.17ms**

id	name	balance
1	Alice	9800.0
2	Bob	7200.0

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Atomicity guarantees that a money transfer happens completely or not at all. If any part of the transaction fails, MySQL rolls back all changes so no money is lost or partially transferred. Rollback is the particular property through which we can go back to previous state and never lost in the middle of a transfer.

TASK 2 Joins

Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

```
SELECT
c.id,
c.name,
o.id AS order_id,
o.total,
o.status
FROM customers c
INNER JOIN orders o
ON c.id = o.customer_id;
```

Output:

Results

[Copy as Markdown](#)

Query #1 **Execution time: 0.49ms**

id	name	order_id	total	status
1	Rahul	1	450.0	completed
1	Rahul	18	510.0	completed
1	Rahul	35	950.0	completed
2	Amit	2	250.0	pending

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

```
SELECT
c.id,
c.name,
o.id AS order_id,
o.total
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id;
```

```
SELECT
c.id,
c.name
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id
WHERE o.id IS NULL;
```

Output:

17	Rakesh	17	900.0
17	Rakesh	34	700.0
18	Sonal	null	null
19	Bhavik	null	null
20	Jay	null	null

Query #2 Execution time: 0.2ms

id	name
18	Sonal
19	Bhavik
20	Jay

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

A marketing team prefers the LEFT JOIN result because it includes customers who have never placed an order, allowing them to target those customers with promotional offers and it make it useful for company.

TASK 3 Indexing

Using your orders table:

- Pick a column that is not currently indexed (for example status or created_at).
- Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status='completed';
```

Output:

Query #1 Execution time: 0.45ms

EXPLAIN

-> Filter: (orders.`status` = 'completed') (cost=4.25 rows=4) (actual time=0.0301..0.0439 rows=31 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0227..0.0293 rows=40 loops=1)

c) Create an index on that column.

```
CREATE INDEX idx_status
ON orders(status);
```

Output:

Query #2 Execution time: 25.06ms

There are no results to be displayed.

d) Re-run the exact same query and record the new numbers.

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE status='completed';
```

Output:

Query #3 Execution time: 0.59ms

EXPLAIN

-> Index lookup on orders using idx_status (status = 'completed') (cost=3.85 rows=31) (actual time=0.0896..0.0951 rows=31 loops=1)

e) Submit a one-line comparison of before vs. after.

After creating the index, MySQL can locate matching rows more efficiently, reducing the amount of data it needs to scan and it also reduces the time considerably and making it efficient.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT *
FROM orders
WHERE customer_id IN
(
SELECT id
FROM customers
WHERE signup_date >= '2024-02-01'
);
```

Output:

Results

There are no results to be displayed.

a) Rewrite it as a JOIN instead of a subquery.

```
SELECT *
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.signup_date >= '2024-02-01';
```

Output:

Query #1 Execution time: 0.55ms

id	customer_id	restaurant_id	total	status	created_at
6	6	6	900.0	completed	2024-04-01 13:30:00
23	6	2	450.0	completed	2024-04-03 11:00:00
40	6	3	520.0	completed	2024-04-04 13:40:00
7	7	7	420.0	pending	2024-04-01 14:00:00
24	7	1	800.0	completed	2024-04-03 11:30:00
8	8	8	700.0	completed	2024-04-01 14:20:00
25	8	10	520.0	completed	2024-04-03 12:00:00
9	9	9	380.0	completed	2024-04-01 14:40:00
26	9	9	250.0	completed	2024-04-03 12:20:00
10	10	10	550.0	completed	2024-04-01 15:00:00

Query #2 Execution time: 0.43ms

id	customer_id	restaurant_id	total	status	created_at	id	name	email	signup_date
6	6	6	900.0	completed	2024-04-01 13:30:00	6	Riya	riya@gmail.com	2024-02-01
23	6	2	450.0	completed	2024-04-03 11:00:00	6	Riya	riya@gmail.com	2024-02-01
40	6	3	520.0	completed	2024-04-04 13:40:00	6	Riya	riya@gmail.com	2024-02-01
7	7	7	420.0	pending	2024-04-01 14:00:00	7	Sneha	sneha@gmail.com	2024-02-05
24	7	1	800.0	completed	2024-04-03 11:30:00	7	Sneha	sneha@gmail.com	2024-02-05
8	8	8	700.0	completed	2024-04-01 14:20:00	8	Vivek	vivek@gmail.com	2024-02-10
25	8	10	520.0	completed	2024-04-03 12:00:00	8	Vivek	vivek@gmail.com	2024-02-10

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT*).

```
SELECT
o.id,
c.name,
o.total,
o.status,
o.created_at
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.signup_date >= '2024-02-01';
```

Output:

Query #3 Execution time: 0.37ms

id	name	total	status	created_at
6	Riya	900.0	completed	2024-04-01 13:30:00
23	Riya	450.0	completed	2024-04-03 11:00:00
40	Riya	520.0	completed	2024-04-04 13:40:00
7	Sneha	420.0	pending	2024-04-01 14:00:00
24	Sneha	800.0	completed	2024-04-03 11:30:00
8	Vivek	700.0	completed	2024-04-01 14:20:00
25	Vivek	520.0	completed	2024-04-03 12:00:00
9	Ankit	380.0	completed	2024-04-01 14:40:00
26	Ankit	250.0	completed	2024-04-03 12:20:00

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Original Query

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id IN
(
SELECT id
FROM customers
WHERE signup_date >= '2024-02-01'
);
```

Optimized query:

```
EXPLAIN ANALYZE
SELECT
  o.id,
  c.name,
  o.total,
  o.status,
  o.created_at
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.signup_date >= '2024-02-01';
```

Output:

Query #4 Execution time: 0.38ms

EXPLAIN

-> Nested loop inner join (cost=4.58 rows=6.67) (actual time=0.0178..0.0733 rows=25 loops=1) -> Filter: (customers.signup_date >= DATE'2024-02-01') (cost=2.25 rows=6.67) (actual time=0.00887..0.0156 rows=15 loops=1) -> Table scan on customers (cost=2.25 rows=20) (actual time=0.00657..0.0125 rows=20 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.265 rows=1) (actual time=0.003..0.0035 rows=1.67 loops=15)

Query #5 Execution time: 0.3ms

EXPLAIN

-> Nested loop inner join (cost=4.58 rows=6.67) (actual time=0.02..0.0749 rows=25 loops=1) -> Filter: (c.signup_date >= DATE'2024-02-01') (cost=2.25 rows=6.67) (actual time=0.0107..0.0181 rows=15 loops=1) -> Table scan on c (cost=2.25 rows=20) (actual time=0.00855..0.0153 rows=20 loops=1) -> Index lookup on o using customer_id (customer_id = c.id) (cost=0.265 rows=1) (actual time=0.00298..0.00347 rows=1.67 loops=15)

The optimized query uses a JOIN instead of a subquery and selects only the required columns instead of SELECT *. This can reduce unnecessary data retrieval and improve query readability and performance.

Final Challenge — A Real-World Problem

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

```
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
INNER JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Output:

Results

Query #1 Execution time: 0.34ms

restaurant_name	total	status	created_at
Royal Biryani	950.0	completed	2024-04-04 12:00:00
Spice Villa	510.0	completed	2024-04-02 13:20:00
Pizza Hub	450.0	completed	2024-04-01 12:00:00

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

```
EXPLAIN ANALYZE
SELECT
    r.name AS restaurant_name,
    o.total,
    o.status,
    o.created_at
FROM orders o
INNER JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Output:

Query #2 Execution time: 0.41ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.024..0.0279 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.023..0.0266 rows=3 loops=1) -> Sort: o.created_at DESC (cost=1.05 rows=3) (actual time=0.0188..0.0191 rows=3 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=0.0115..0.0132 rows=3 loops=1) -> Index lookup on o using customer_id (customer_id = 1) (cost=1.05 rows=3) (actual time=0.0109..0.0123 rows=3 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.283 rows=1) (actual time=0.00191..0.00196 rows=1 loops=3)

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

```
CREATE INDEX idx_customer_created
ON orders(customer_id, created_at);
```

Output:

Query #3 Execution time: 10.06ms

There are no results to be displayed.

d) Re-run the same query and compare the new numbers to step (b).

```
EXPLAIN ANALYZE
SELECT
  r.name AS restaurant_name,
  o.total,
  o.status,
  o.created_at
FROM orders o
INNER JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Output:

Query #4 Execution time: 0.54ms

EXPLAIN

-> Limit: 5 row(s) (cost=2.1 rows=3) (actual time=0.0396..0.046 rows=3 loops=1) -> Nested loop inner join (cost=2.1 rows=3) (actual time=0.0387..0.0448 rows=3 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.05 rows=3) (actual time=0.0308..0.0335 rows=3 loops=1) -> Index lookup on o using idx_customer_created (customer_id = 1) (reverse) (cost=1.05 rows=3) (actual time=0.0297..0.0321 rows=3 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.283 rows=1) (actual time=0.00315..0.0032 rows=1 loops=3)

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

```
SELECT
  r.name AS restaurant_name,
  o.total,
  o.status,
  o.created_at
FROM orders o
INNER JOIN restaurants r
ON o.restaurant_id = r.id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC
LIMIT 5;
```

Output:

Query #5 Execution time: 0.25ms

restaurant_name	total	status	created_at
Royal Biryani	950.0	completed	2024-04-04 12:00:00
Spice Villa	510.0	completed	2024-04-02 13:20:00
Pizza Hub	450.0	completed	2024-04-01 12:00:00

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Before Index execution time

0.34 ms

After Index execution time

0.25 ms

The composite index on (customer_id, created_at) helps MySQL quickly locate the orders for a specific customer and retrieve them in the required sort order. This reduces the amount of data that needs to be scanned and improves query performance, especially as the orders table grows larger.

As a result, the customer profile page loads faster and uses database resources more efficiently. This optimization is particularly beneficial in real-world applications where thousands of orders are stored and users expect quick response times.